Appendix: Artifact Description/Artifact Evaluation

Artifact Description (AD)

I. Overview of Contributions and Artifacts

A. Paper's Main Contributions

In this paper we explore the use of off-the-shelf LLMs for estimating the Roofline Arithmetic Intensity (RAI) of CUDA and OpenMP GPU kernels. The contributions of our paper are as follows:

- C_1 We create an LLM benchmark consisting of source code features, SASS instructions, and profiling data of 254 GPU kernels across four NVIDIA GPUs (RTX 3080, A10, A100, H100). These codes were sourced from the `HeCBench` suite listed in Table II.
- C_2 We query three off-the-shelf large language models (LLMs) (`GPT-OSS`, `GPT-5.4`, `Opus-4.6`) with a custom querying pipeline, comparing the models' ability to predict the FP16/32/64 Roofline Arithmetic Intensity (RAI) of the target sampled kernels across OpenMP and CUDA runtimes. This querying pipeline offers a hardware-free approach to estimating GPU kernel performance, enabling efficient triage during development before a kernel performs potentially costly runs on real hardware.
- C_3 Our comparisons of frontier and open-source LLMs find that the premium/closed-source/frontier `Opus-4.6` and `GPT-5.4` LLMs produce the most accurate RAI predictions overall when prompting with only source code, whereas the inexpensive/open-source `GPT-OSS` model struggles in comparison.
- C_4 We discover that prompting with post-compilation artifacts such as SASS, allow the LLMs to greatly improve prediction accuracy across GPUs due to the added knowledge of FLOPs and read/write operations that get added/removed by the compiler. This effect is very apparent when the compiler adds implicit FP16 operations that are not visible from source alone.
- C_5 We find that there exists a bias across all three LLMs, where they are more easily able to predict the RAI of GPUs with larger L2 caches. This is due to how the larger caches enable less GPU RAM read/write ops, minimizing operations that the LLMs must account for. This bias is most apparent on the 3080 and A10 GPUs, which sport smaller L2 caches that lead to more memory read/writes that the LLMs fail to account for.
- C_6 We find that source codes with floating-point operations that introduce/remove FLOPs (e.g: FLOP division, float sub-expressions, transcendental math functions) from the final compiled kernel have a higher chance of causing LLM RAI prediction errors. This is due to how the LLMs must reason more deeply about the provided source/SASS and how it maps to the computation being performed.
- C_7 We show that although the LLMs suffer from RAI estimate errors, when we use them to categorize kernels as either bandwidth-bound or compute-bound, the RAI estimates are very well aligned with the ground-truth categorizations, implying that LLMs can offer actionable insights as to the behavior of the sampled kernels.
- C_8 We identify cost and query time tradeoffs between the models, noting how the best-performing `Opus-4.6` model has high query response times and is the most expensive to query, while the worst-performing `GPT-OSS` model is the cheapest to query with worst response times, and the `GPT-5.4` offers a good middle-ground of cost with the fastest query times.

B. Computational Artifacts

The computational artifact associated with this paper is:

A_1 https://doi.org/10.5281/zenodo.19866667,

which is a snapshot of our Github repository https://github.com/FLOPBench/SC26FLOPBench that contains all the files necessary to recreate/run our experiments. Given that we depend on particular GPU hardware and an LLM-query API, our Zenodo snapshot contains various database dumps of all our files including profiling data and LLM query responses. Below is a table mapping each of the previously-mentioned contributions to their respective paper figures, listings, and tables.

Artifact	Contribution	Paper Elements
A_1	C_1	Figure 2
	C_2	Listings 1–3
	C_3, C_4, C_5	Figures 6, 7; Table 3
	C_6	Figure 8
	C_7	Figure 7; Table 3
	C_8	Figures 9, 10

The following is a high-level directory structure of A_1:

- **`HeCBench/`** — Git submodule containing the full `HeCBench` benchmark suite, including all CUDA and OpenMP source directories and their associated build scripts.
- **`cuda-profiling/`** — Profiling infrastructure for executing benchmarks with Nsight Compute and collecting per-kernel roofline metrics. Contains the `gatherData.py` profiling script and the `collected-data/` subdirectory, which holds the pre-collected per-GPU profiling archives, the SASS disassembly archive, and helper scripts for extracting and condensing the raw `.ncu-rep` reports.

- **dataset-creation/** — Scripts that merge profiling metrics, SASS disassembly, and scraped source code into the unified `gpuFLOPBench.json` benchmark dataset consumed by all LLM experiment pipelines.
- **experiments/** — LLM querying pipelines and analysis scripts, organized into three subdirectories: `direct-prompting/` (RAI estimation via LangGraph), `feature-voting/` (static code-feature classification), and `error-analysis/` (Cliff's delta feature-to-error association).
- **unit-tests/** — `pytest` test suite verifying build artifacts, kernel extraction, demangling, dataset integrity, and SHA-256 hashes of all generated paper outputs.
- **reproduce_paper_results.sh** — Convenience script that automates the full reproduction workflow: database restoration from the included dumps, figure and listing generation, and artifact evaluation tests.
- **Dockerfile** — Container definition providing a reproducible build and LLM-querying environment based on the NVIDIA HPC SDK image with CUDA 13.0.

II. Artifact Identification

A. Computational Artifact A_1

Relation To Contributions

This artifact contains all the source code necessary to build, execute, profile, and source/SASS scrape all 254 CUDA kernels examined by our work (C_1). It also contains all the scripts to run LLM queries, collect query metadata, and produce the figures, listings, and results table present in this work (C_2 through C_8).

Expected Results

The provided artifact reproduces all the experiments and results that we report in our work. A user should expect to successfully reproduce all figures, tables, and prompt listings reported in this work using the pre-collected profiling data and database dumps, without requiring access to GPU hardware or live LLM API credits.

In the event that a user does not have access to a GPU or LLM to recreate our experiments, we provide raw dumps of our databases containing our LLM query results. We note that for users running their own LLM queries, they may not get the same exact results that we get, however the general trends of C_{3-8} should be very similar.

Expected Reproduction Time (in Minutes)

The total time to perform all the code building/profiling, and LLM queries is about 28 hours. Table I shows a breakdown of how much time is dedicated to each of the various tasks needed for reproducing this work.

Artifact Setup (incl. Inputs)

Hardware: To reproduce our profiling results, we expect a user to have access to machines using the four GPUs listed in Table 1 of the paper, an NVIDIA RTX 3080, A10, A100, and H100 GPUs. All our profiled programs were single-node codes that execute on a single GPU. We also made sure that no other GPU programs were being executed when profiling our 254 codes, this included making sure that the `nvidia-smi` reported an idle memory usage of `0 MiB` to guarantee that the profiled GPU DRAM reads/writes were not polluted by other co-executing programs (e.g.: Ubuntu display driver).

TABLE I
ESTIMATED AMOUNT OF TIME REQUIRED FOR EACH OF THE STEPS IN REPRODUCING OUR WORK.

Phase	Task Description	Time
Setup	Source Compilation	≈ 30 mins
Setup	Source Scraping	≈ 10 mins
Setup	SASS Scraping	≈ 10 mins
Setup	Dataset Compilation	≈ 10 mins
Execution	GPU Profiling	≈ 5 hours
Execution	LLM-based Code Feature Voting	≈ 12 hours
Execution	LLM RAI Estimation Querying	≈ 12 hours
Analysis	Plot Generation	≈ 10 mins

TABLE II
SOFTWARE VERSIONS USED IN THIS WORK.

Package Name	Version	URL
OS Version	Ubuntu 24.04	https://ubuntu.com
CUDA Toolkit	13.0	https://developer.nvidia.com/cuda-toolkit
CUDA Driver	580.65.06	https://developer.nvidia.com/cuda-downloads
`HeCBench`	`4c1aee3b`	https://github.com/zjin-lcf/HeCBench
LLVM Compiler	21.1.8	https://llvm.org
`nvcc` Compiler	13.0	https://developer.nvidia.com/hpc-sdk
Python Interpreter	3.11	https://www.python.org
CMake	3.28.3	https://cmake.org
PostgreSQL	16	https://www.postgresql.org
`ncu` Profiler	2025.3.1.0	https://developer.nvidia.com/nsight-compute
LangChain	1.2.12	https://github.com/langchain-ai/langchain
LangGraph	1.1.1	https://github.com/langchain-ai/langgraph

Software: We made sure that the OS version, CUDA GPU driver/library versions, and compiler versions were the same across tested hardware. Table II lists the various software packages and version numbers used in this work.

All 254 kernels that we sample are sourced from programs found in the `HeCBench` suite, which is a repository that contains a collection of CUDA, OpenMP, SYCL, and HIP codes. With respect to kernel compilation, we build all our target programs with their default `HeCBench` compilation parameters, which use the `-O3` optimization flag. We use the `HeCBench` default input arguments to execute each program, using NVIDIA's `ncu` profiler to capture runtime metrics of the first execution of each of the sampled kernels.

Datasets / Inputs: Our artifact is capable of generating all the datasets needed in this work, however we list the intermediate and final datasets for reference. All large dataset files are included directly in the Zenodo archive (A_1) and require no additional download step.

- **gpuFLOPBench.json** — The primary benchmark dataset consolidating profiling metrics, SASS disassembly, and scraped source code for all 254 sampled GPU kernels. Included in the Zenodo archive file `gpuFLOPBench.json`; serves as the primary input to all LLM experiment scripts, containing the pro-

filed/scraped GPU kernels with their metrics, source code, and SASS instructions.

- **Per-GPU Profiling Archives** — Four archives (`NVIDIA_*_profiling-results-*.zip`) containing Nsight Compute `.ncu-rep` report files collected from each of the four target GPUs. Included in the Zenodo archive under `cuda-profiling/collected-data/` and extracted with the provided `unzip_collected_data.py` helper script. These `.ncu-rep` files are scraped for the appropriate Roofline metrics and used to form the `gpuFLOPBench.json` benchmark file.
- **SASS Disassembly Archive** — A zip archive of SASS disassembly extracted from all compiled executables, included in the Zenodo archive in `sass_files.zip`. Extracted by the same helper script alongside the profiling archives. These `*.sass` files contain the SASS instructions for all the kernels corresponding to each profiled executable on each GPU architecture.
- **`gpuflops_db.dump`** — A PostgreSQL dump of all direct-prompting LLM RAI estimation query results, included in the Zenodo archive in the `gpuflops_db.dump` file. Can be restored without re-running LLM queries; see the `README.md` for restore instructions. This LLM query response data is used to calculate the RAI errors for Table 3 and Figures 6 and 7.
- **`code_features_db.dump`** — A PostgreSQL dump of all code-feature voting LLM query results, included in the Zenodo archive in the `code_features_db.dump` file. Can be restored without re-running LLM queries; see the `README.md` for restore instructions. This code-feature voting data is the result of the ensemble LLM voting that is used to determine what code features are present in each of the kernels. This data is ultimately used to make Figure 8 that associates the code features that frequently appear alongside the RAI prediction errors.
- **`request_metadata.dump`** — A PostgreSQL dump of per-request OpenRouter cost and timing metadata used to produce Figures 9 and 10, included in the Zenodo archive in the `request_metadata.dump` file. Can be restored without OpenRouter API access; see the `README.md` for restore instructions. This data was sourced post-LLM-queries as the initial OpenRouter LLM query responses do not contain accurate cost and execution time metadata.

Installation and Deployment: For users who only wish to reproduce the paper's figures, tables, and listings, the minimal setup path is to unzip the Zenodo archive (A_1), install the Python dependencies via `pip install -r requirements.txt`, and run the top-level `reproduce_paper_results.sh` convenience script. This script restores all included database dumps, generates all paper figures and listings, and runs the artifact evaluation tests. No GPU hardware or live LLM API credentials are required for this path.

For users who wish to gather new GPU profiling data or extend the `gpuFLOPBench.json` dataset with additional GPUs, a more complete environment is required. The `Dockerfile` in A_1 provides a fully configured container based on the NVIDIA HPC SDK image, and the `README.md` describes how to build and run it across different host platforms (Linux, macOS, Windows). For users seeking access to the exact GPU hardware used in this work, the `README.md` also contains step-by-step instructions for configuring a *Lambda.ai*[1] cloud instance with the correct CUDA driver and compiler versions. Given that these instructions include system-level driver changes, we recommend using a cloud instance rather than a personal workstation.

The next section details the high-level tasks involved in the full end-to-end workflow.

Artifact Execution

The following is a list of tasks required to generate the results of this paper. We break them down into two groups: (1) GPU profiling and source scraping, and (2) LLM querying. Detailed instructions for each task are provided in the corresponding sections of the `README.md`.

The GPU profiling of phase (1) includes the following steps:

- T_1: Build all `HeCBench` CUDA and OpenMP benchmarks. See the *Build Benchmarks* section of the `README.md`.
- T_2: Profile each target GPU with Nsight Compute, repeated independently on each of the four GPU systems (RTX 3080, A10, A100, H100). Produces per-GPU profiling archives and summary CSVs. See the *Profile Benchmarks* section of the `README.md`. Depends on T_1.
- T_3: Extract SASS disassembly from the compiled executables, scrape benchmark source files, and merge the profiling data, SASS, and source into the unified `gpuFLOPBench.json` dataset. See the *Phase 2 — Build the gpuFLOPBench Dataset* section of the `README.md`. Depends on T_1 and T_2.

For phase (2), the LLM querying, we break it down into the following tasks:

- T_4: Run or restore the two LLM query experiments: the code-feature voting pipeline and the direct-prompting RAI estimation pipeline. Users who wish to skip re-running live LLM queries may restore the included database dumps instead. See the *Phase 3 — LLM Experiments* section of the `README.md`. Depends on T_3 (or the included database dumps as a substitute).
- T_5: Generate all paper figures (Figures 2, 6–10), Table 3, and Listings 1 and 3 from the populated databases. See the *Phase 4 — Paper Figures, Tables & Listings* section of the `README.md`. Depends on T_4.

The full dependency chain for a complete reproduction from source is $T_1 \rightarrow T_2 \rightarrow T_3 \rightarrow T_4 \rightarrow T_5$.

[1] https://lambda.ai

TABLE III
SCRIPTS (UNDER EXPERIMENTS/) AND THEIR PAPER OUTPUTS.

Script	Paper Elements
direct-prompting/ make_plots_for_paper.py	Figs. 2, 6, 7; Table 3
direct-prompting/ fetch_openrouter_request_metadata.py	Figs. 9, 10
error-analysis/ make_plots_for_paper.py	Fig. 8
direct-prompting/ print_prompt_for_paper_listing_1.py	Listings 1, 3

Users who rely on the included datasets and database dumps may skip directly to T_4 (restore) and T_5, or use the `reproduce_paper_results.sh` convenience script which automates both steps end-to-end.

Artifact Analysis (incl. Outputs)

All paper figures, tables, and listings are generated by the scripts in the `experiments/` directory, using the populated PostgreSQL databases as input. Table III lists each script alongside its corresponding paper elements. Using the pre-collected database dumps included in the Zenodo archive, the resulting figures and tables should exhibit the same trends across the three sampled LLMs. We note that Listing 2 (the XML input prompt example in the appendix) is embedded directly in the paper's appendix source rather than being auto-generated. Figures 9 and 10 are reproduced from the included `request_metadata.dump`, which contains all per-request cost and timing metadata. See the *Phase 4 — Paper Figures, Tables & Listings* section of the `README.md` for full invocation instructions.